JULIAN KRAUSS
ITA-09/25 · PROGRAMMIERTECHNIK

VIER GEWINNT

GYM EDITION

Objektorientiertes C++17-Konsolenspiel

Projektvorstellung und Live-Code in Visual Studio Code

15 MIN PRÄSENTATION | **15 MIN CODE-DEMO**



© 2026 Julian Krauß · JK-CD-DE-2026-003

02 / AGENDA / INHALTSVERZEICHNIS

Was Sie heute erwartet

Eine strukturierte Projektvorstellung und anschließend die Demonstration direkt im Quellcode.

01 **Projektidee & Anforderungen**
Ziel, Spielregeln, flexible Feldgrößen
Folien 03–04

02 **Spielablauf & Objektmodell**
Zustände, Game, Board, Player und Move
Folien 05–06

03 **Codeaufbau & Sieglogik**
Header/CPP und vier Prüfungsachsen
Folien 07–08

04 **Persistenz & Komfortbefehle**
Speichern, Laden, Sonderbefehle
Folien 09–10

05 **Tests, Projektstand & Fazit**
Erreichte Funktionen, offene Nachweise
Folien 11–13

06 **Live-Code in Visual Studio Code**
Anschließend 15 Minuten ausführbarer Code
Folie 14

TEIL A · 15 MIN PROJEKTPRÄSENTATION | TEIL B · 15 MIN CODE + DEMO

03 / PROJEKTIDEE

Klassisches Spiel. Technische Neuinterpretation.

Das bekannte Spielprinzip bleibt erhalten. Die Umsetzung setzt auf C++17 und objektorientierte Programmierung.

PROJEKT | SPIELABLAUF + OOP | ALGORITHMEN | TEST + STATUS | LIVE-CODE

Spielfunktion

Zwei bis vier Personen spielen lokal in der Konsole. Vier gleiche Symbole in Folge entscheiden die Partie.

Gym-Konzept

Die üblichen Spielsteine erscheinen in der Terminalausgabe als ASCII-Kurzhanteln, z. B. =[R]=.



VISUALISIERUNG · DESIGN-MOCKUP · KEINE GUI

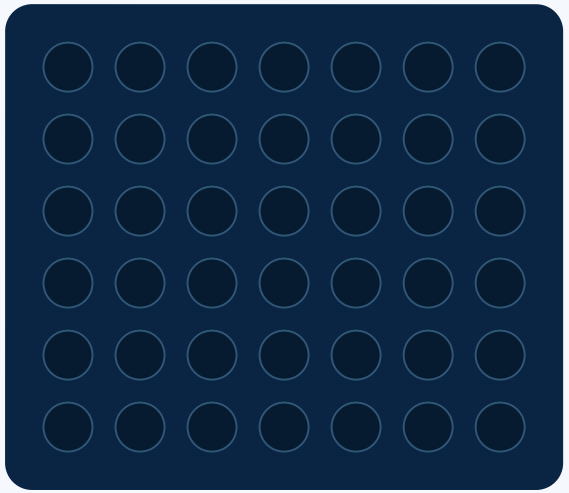
04 / ANFORDERUNGEN

Ein Spielprinzip. Drei Spielfeldgrößen.

Je mehr Personen teilnehmen, desto größer ist das Spielfeld. Die Gewinnbedingung bleibt gleich.

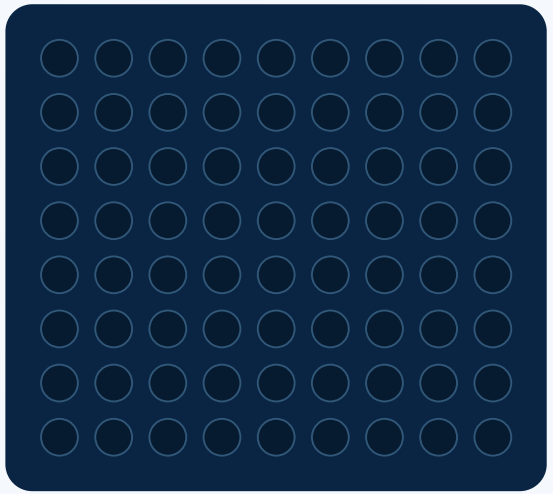
PROJEKT | SPIELABLAUF + OOP | ALGORITHMEN | TEST + STATUS | LIVE-CODE

2 SPIELER



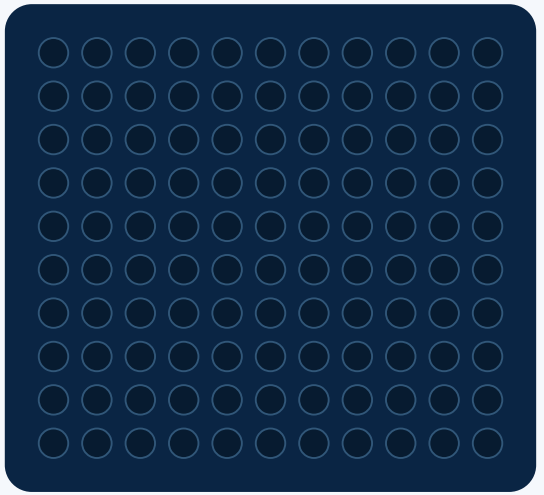
7 × 6

3 SPIELER



9 × 8

4 SPIELER



11 × 10

Gewinnregel 4 in einer Reihe

Ungültige Eingaben werden abgefangen · Nur ein gültiger Spielzug wechselt den Spieler.

05 / ABLAUF

Vom Startmenü bis zum Ergebnis

Jeder gültige Zug führt durch eine definierte Folge von Zustandsänderungen und Prüfungen.

PROJEKT | SPIELABLAUF + OOP | ALGORITHMEN | TEST + STATUS | LIVE-CODE

01
Konfigurieren
Spielerzahl und eindeutige Namen erfassen

›

02
Stein einwerfen
Spalte validieren und freie Zeile bestimmen

›

03
Ergebnis prüfen
Sieg, Remis oder nächsten Spieler auswählen

›

04
Fortsetzen
Nächster Zug oder Rückkehr zum Menü

Zentrale Regel Eine ungültige Eingabe erzeugt keinen Spielzug und keinen Spielerwechsel.

06 / ARCHITEKTUR

Vier Typen. Klare Verantwortlichkeiten.

Die Daten und Aufgaben sind in einzelne C++-Typen gegliedert.

Game

Steuert das Hauptmenü und den Spielablauf.
Verwaltet aktive Spieler, Züge, Spielstände und Highscores.

Board

Raster, Spielzüge und Gewinnprüfung

Player

Name und eindeutiges Symbol

Move

Zeile · Spalte · Spielerindex für die Rückgängig-Historie

1 Board · 2–4 Player · 0..* Move

07 / C++-STRUKTUR

Quelltext getrennt nach Verantwortung

Header deklarieren Schnittstellen, CPP-Dateien enthalten die zugehörige Implementierung.

PROJEKTSTRUKTUR

```
vier-gewinnt-gym-edition/
  include/
    Player.h
    Board.h
    Game.h
  src/
    main.cpp
    Board.cpp
    Game.cpp
```

```
Board::Board(int playerCount)
 : rows_(6 + 2 * (playerCount - 2)),
   columns_(7 + 2 * (playerCount - 2)),
   grid_(rows_, std::vector<char>(columns_, '.')) {}
```

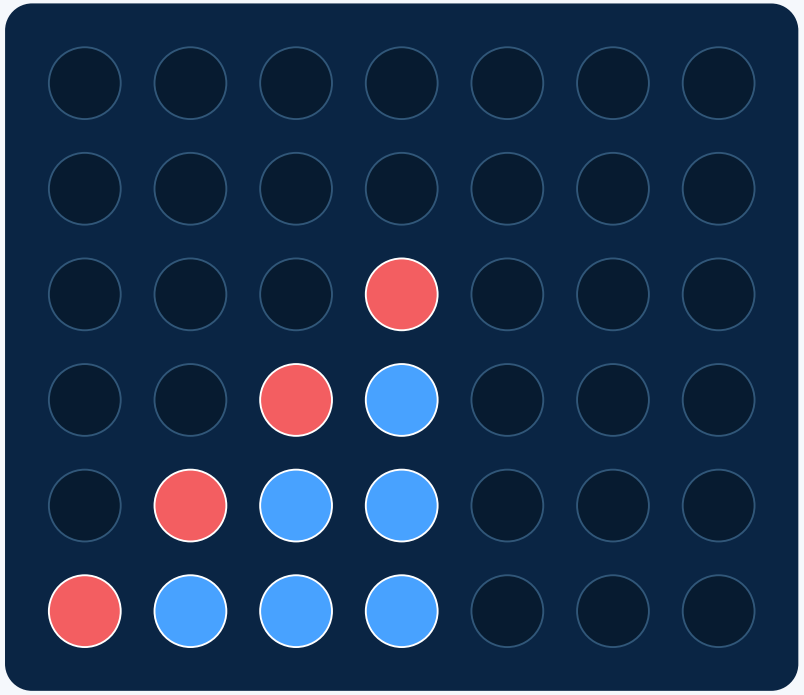
EINE REGEL, ZENTRAL DEFINIERT

Neue Spielfeldgrößen werden im Konstruktor berechnet. Alle Spielmodi nutzen danach dieselbe Board-Logik.

08 / ALGORITHMUS

Gewinnprüfung in vier Richtungsachsen

Nach jedem gültigen Zug wird vom letzten Stein aus in beide Richtungen gezählt.



BEISPIEL: DIAGONALE VIERERREIHE

Horizontal · Vertikal · Diagonal ↘ · Diagonal ↙

```
int length = 1
  + countDirection(...,  dr,  dc, symbol)
  + countDirection(..., -dr, -dc, symbol);
if (length >= 4) return true;
```

Nur vier Achsen nötig: jede deckt ihre Gegenrichtung mit ab.

09 / PERSISTENZ

Spielstände sichern und kontrolliert laden

Die Partie wird lokal als Text gespeichert. Beim Laden wird zuerst geprüft, danach übernommen.

SAVEGAME-FORMAT

```
GYM1
2 1 6 7 3
"Julian" R
"Daniel" B
... Zugliste ...
... Spielfeld ...
```

Schematischer Ausschnitt · Formatversion + Zustand

01

Einlesen

Signatur, Dimensionen, Namen und Züge prüfen

02

Rekonstruieren

Temporäres Board für gültige Spielzüge

03

Übernehmen

Erst nach Validierung echten Zustand ersetzen

Fehlerhafte Dateien → Laden ablehnen, Spielzustand bleibt erhalten.

10 / FUNKTIONEN

Mehr als nur Steine setzen

Ergänzende Konsolenbefehle verbessern die Bedienbarkeit, ohne die Regeln zu verändern.

V 4 — **Spaltenvorschau**
nur Anzeige

U — **Letzten Zug zurücknehmen**
Zughistorie

S — **Spielstand speichern**
lokale Datei

H — **Hilfetext**
Kurzanleitung

I — **Partiestatistik**
Füllstand & Spieler

A — **Fallanimation an/aus**
ANSI im Terminal

Die 3D-Oberfläche ist ein Konzept. Die ausführbare Ausgabe erfolgt im Terminal.

11 / QUALITÄTSSICHERUNG

Was ist tatsächlich getestet?

Automatisierte Tests sind von noch offenen manuellen Abnahmetests zu unterscheiden.

AUTOMATISCH VERIFIZIERT

- Raster 7×6, 9×8 und 11×10
- Ungültige Spalten abweisen
- Gewinn waagerecht und senkrecht
- Beide diagonalen Gewinnrichtungen
- Undo auf Board-Ebene

MANUELL NOCH NACHWEISEN

- Vollständiger Partiedurchlauf
- Beschädigte Spielstandsdatei
- Highscore nach Neustart
- GitHub-Commit-Historie

Prüfgrundlage: tests/board_tests.cpp · docs/TESTPLAN.md

12 / PROJEKTSTAND

Realisierte Funktion und Designidee getrennt halten

Der aktuelle Stand ist ein C++17-Konsolenspiel, kein fertig ausgeliefertes grafisches Spiel.

IM QUELLCODE VORHANDEN

- Klassen Game, Board, Player, Move
- Spielmodus mit 2 bis 4 Personen
- Gewinnprüfung und Zugvalidierung
- Speichern, Laden und Highscore
- Konsolenausgabe mit Kurzhanteln

NICHT ALS PRODUKT IMPLEMENTIERT

- Echte 2D-/3D-Fensteroberfläche
- Grafische Schaltflächen / Maussteuerung
- Hardwarebeschleunigter Renderer
- Online-Multiplayer oder KI-Gegner
- Vollständige Schulabnahme

© 2026 Julian Krauß · JK-CD-DE-2026-003 · Bernd-Blindow-Schulen Ravensburg

13 / FAZIT

Die Architektur ist die eigentliche Leistung.

Das Projekt verbindet objektorientierte Modellierung, Regelalgorithmen und sichere Dateiverarbeitung.

PROJEKT | SPIELABLAUF + OOP | ALGORITHMEN | TEST + STATUS | LIVE-CODE

01 Struktur

Verantwortlichkeiten sind klar in Game, Board, Player und Move aufgeteilt.

02 Logik

Ein zentraler Algorithmus prüft jede mögliche Gewinnrichtung.

03 Robustheit

Ungültige Eingaben und Dateifehler sind im Entwurf berücksichtigt.

NÄCHSTER SCHRITT → QUELLCODE LIVE IM EDITOR

© 2026 Julian Krauß · JK-CD-DE-2026-003 · Bernd-Blindow-Schulen Ravensburg

14 / LIVE-CODE / AGENDA

Jetzt in Visual Studio Code.

15 Minuten · Programmstart, Klassen, Sieglogik und ausführbare Tests.

00–02	02–05	05–09	09–12	12–15
main.cpp & Game.h	Board-Konstruktor & Spielfeld	drop() & hasWon()	Game::play() & Speichern	Build, Test und Demo

Terminal im Projektordner öffnen · Animation für die Live-Vorführung bei Bedarf mit „A“ deaktivieren

15 / RESERVE / RÜCKFRAGEN

Code-Navigation auf einen Blick

Bei Detailfragen lassen sich die Funktionen in den zugehörigen Dateien direkt zeigen.

`main.cpp`	Game-Objekt erstellen, run() starten
`include/Game.h`	Spielfluss, Spielerdaten, Speicherfunktionen
`include/Board.h`	Öffentliche Board-Schnittstelle
`src/Board.cpp`	drop(), hasWon(), undo(), animateDrop()
`src/Game.cpp`	Menü, Sonderbefehle, Dateivalidierung
`tests/board_tests.cpp`	Automatische Board-Prüfungen

© 2026 Julian Krauß · JK-CD-DE-2026-003 · Bernd-Blindow-Schulen Ravensburg

16 / RESERVE / FACHFRAGEN

Drei typische Fragen zur Implementierung

Kurz, präzise und mit nachprüfbarem Codebezug antworten.

Warum vier Achsen?

Jede Achse wird in beide Richtungen gezählt. Weitere Richtungen wären redundant.

Warum eine temporäre Board-Instanz beim Laden?

Fehlerhafte Dateien verändern so nicht den laufenden Spielzustand.

Warum enthält Player keinen Highscore?

Die vorliegende Umsetzung hält Siege zentral in Game::highscores_ (map<string,int>).

© 2026 Julian Krauß · JK-CD-DE-2026-003 · Bernd-Blindow-Schulen Ravensburg

17 / RESERVE / QUELLEN

Dokumentationsbasis und Status

Folien basieren auf den bereitgestellten Projektdokumenten und dem Quelltextstand vom 22./23.10.2026.

Gesamtdokumentation	UI-Konzept, Lernmodule, technische Kapitel und zwölf PAP
Anforderungsanalyse	Pflichtfunktionen, Abnahmekriterien und Projektgrenzen
UML-Klassendiagramm	Vier Typen, Beziehungen und Zuordnung zu den Headern
C++17-Projektarchiv	src/, include/, tests/ und lokaler Konsolen-Build

Build/Test extern geprüft am 25.10.2026; vollständige Schulabnahme und manuelle Testfälle nicht attestiert.